

Trade Bot Advisor v3.5

Signal-First Architecture

Product Documentation

Date: 2026-04-30

URL: <https://huggingface.co/spaces/SutskeverFanBoy/trade-bot-advisor>

DISCLAIMER: This system is for educational and research purposes only.

It does not constitute investment advice.

Table of Contents

1. Executive Summary
2. System Architecture
3. Core Modules
 - 3.1 Signal Engine (signal_engine.py)
 - 3.2 Backtesting Engine (backtester.py)
 - 3.3 Ground Truth Evaluator (evaluator.py)
 - 3.4 Portfolio Simulator (portfolio.py)
 - 3.5 Data Ingestion (data_ingest.py)
 - 3.6 Chart Builder (charts.py)
 - 3.7 LLM Interpreter (llm_interpreter.py)
4. User Interface - 9 Tabs
5. Technical Indicators
6. Signal Generation Rules
7. Risk Management
8. Data Sources & Fallback
9. Supported Assets
10. Performance Metrics
11. Structured Output Format
12. Limitations & Known Issues
13. Future Improvements / Roadmap
14. Tech Stack

1. Executive Summary

Trade Bot Advisor v3.5 is a rule-based algorithmic trading signal generator with a comprehensive evaluation framework. Unlike traditional 'AI trading bots' that rely on LLMs to make trading decisions, this system uses a deterministic signal engine built on proven technical indicators (RSI, SMA crossover, MACD, Bollinger Bands, Volume) with a 5-indicator voting mechanism.

The key architectural decision: LLM is downgraded from 'decision maker' to 'interpreter'. All trading signals come from math-based rules, not language model hallucinations. The LLM only explains WHY a signal was generated - it never generates or overrides signals.

The system includes a full backtesting engine, a ground truth evaluator (model ne dedi? gercek ne oldu?), a stateful portfolio simulator, and an AI trade decision pipeline - all accessible through a 9-tab Gradio UI that always works thanks to sample data fallback.

Key Differentiators

- Signal-First: Deterministic rules, not LLM opinions

- C

2. System Architecture

The system follows a layered architecture with clear separation of concerns:

Layer 1 - Data Ingestion (data_ingest.py):

yfinance (stocks/ETFs) -> Binance REST API (crypto) -> Sample data fallback
Unified OHLCV interface. Never returns None - always provides valid data.

Layer 2 - Signal Engine (signal_engine.py):

OHLCV -> RSI + SMA + MACD + Bollinger + Volume + ATR
-> 5-indicator voting -> BUY/SELL/HOLD
-> Volatility filter -> Confidence scoring -> Position sizing
100% deterministic. No LLM involvement.

Layer 3 - Evaluation (backtester.py + evaluator.py):

Walk-forward backtest with equity tracking
Ground truth: compare each signal vs actual N-day price change

Layer 4 - Portfolio (portfolio.py):

Stateful paper trading with risk limits
Position tracking, PnL, drawdown, win rate

Layer 5 - Presentation (app.py + charts.py):

9-tab Gradio UI with Plotly charts
JSON structured output for every signal

Layer 6 - LLM Interpreter (llm_interpreter.py):

OPTIONAL. Explains signals in natural language.
Falls back to template-based interpretation if no HF_TOKEN.

File Structure

File	Size	Purpose
app.py	40.8 KB	Main Gradio application, 9 tabs, all UI logic
core/signal_engine.py	22.9 KB	Deterministic signal generator, 5 indicators, voting
core/backtester.py	9.4 KB	Walk-forward backtesting engine
core/evaluator.py	7.6 KB	Ground truth evaluator - prediction vs reality
core/portfolio.py	10.8 KB	Stateful portfolio simulator
core/data_ingest.py	5.5 KB	Data fetching: yfinance + Binance + fallback
core/charts.py	11.9 KB	Plotly chart builders (candlestick, RSI, equity, etc.)
core/llm_interpreter.py	5.2 KB	Optional LLM explanation layer

requirements.txt	98 B	Python dependencies
------------------	------	---------------------

3. Core Modules

3.1 Signal Engine (signal_engine.py)

The brain of the system. Generates trading signals using a 5-indicator voting mechanism. Every signal is 100% deterministic - given the same input data, it always produces the same output.

Pipeline: fetch OHLCV -> compute 5 indicators -> vote -> volatility filter -> confidence score -> position size -> structured output

Indicators computed:

1. RSI(14) - Relative Strength Index, momentum oscillator
2. SMA(20/50) Crossover - trend direction (golden cross / death cross)
3. MACD(12,26,9) - momentum and crossover detection
4. Bollinger Bands(20,2) - price relative to statistical bands
5. Volume ratio - current vs 20-day average volume

Voting rules:

- 3+/5 bullish -> BUY | 4+/5 bullish -> STRONG_BUY
- 3+/5 bearish -> SELL | 4+/5 bearish -> STRONG_SELL
- Otherwise -> HOLD
- Extreme volatility + BUY -> downgraded to HOLD (safety filter)

Confidence scoring (0-100): base 50, +25 for 4+ agreement, +15 for 3+, +/-10 for RSI confirmation, +5 for high volume, -15 for extreme volatility.

Position sizing: ATR-based stop-loss and take-profit. Risk per trade: 1% (conservative), 2% (moderate), 4% (aggressive). Max single position: 20% of portfolio.

3.2 Backtesting Engine (backtester.py)

Tests the exact same signal logic against historical data. Walk-forward approach: at each day, generate the signal using only data available at that point, then measure actual outcome after N holding days.

Metrics calculated: accuracy (%), total return (%), Sharpe ratio, max drawdown, profit factor, avg win/loss, buy vs sell accuracy. Compared against buy & hold benchmark.

Configurable parameters: RSI buy/sell thresholds, SMA filter on/off, volatility filter on/off with threshold, holding period (1-30 days), evaluation period (0.5-5 years).

3.3 Ground Truth Evaluator (evaluator.py)

The definitive answer to 'Is this signal engine actually good?' For every historical signal, records what the price actually did over 5, 10, and 20 days.

Output per signal:

- Date, ticker, signal (BUY/SELL), price at signal, RSI at signal
- 5-day outcome: future price, change %, direction (UP/DOWN), correct (yes/no)
- 10-day outcome: same
- 20-day outcome: same

Aggregate metrics per horizon: accuracy, BUY accuracy, SELL accuracy, average directional return, median return, best/worst return.

Calibration analysis: compares accuracy of high-conviction signals (extreme RSI) vs low-conviction signals. Answers: 'Are confident signals more accurate?'

3.4 Portfolio Simulator (portfolio.py)

In-memory stateful portfolio that persists during the session. Tracks open positions, closed trades, PnL, equity snapshots, drawdown.

Risk controls:

- Max 95% cash utilization per trade
- No duplicate positions (can't buy same ticker twice)
- Stop-loss and take-profit tracking
- BUY/STRONG_BUY -> open long | SELL/STRONG_SELL -> close if held | HOLD -> no action

Multi-ticker simulation: runs signal engine on multiple tickers, executes trades, returns full decision table with per-ticker backtest validation.

3.5 Data Ingestion (data_ingest.py)

Three-tier data sourcing with guaranteed fallback:

1. Binance REST API - for crypto tickers (BTC-USD, ETH-USD, etc.)
2. yfinance - for stocks, ETFs, indices, commodities
3. Sample data generator - synthetic OHLCV with realistic random walk

The fallback guarantees the Space ALWAYS works, even when APIs are down or rate-limited. Sample data uses seeded random generation for consistency.

3.6 Chart Builder (charts.py)

All charts built with Plotly (dark theme). Components:

- Candlestick with SMA(20/50) overlay + volume bars
- RSI panel with overbought/oversold zones
- MACD panel with histogram, line, and signal line
- Equity curve with initial capital reference line
- Backtest signal map (triangles on price chart, green=correct, red=wrong)
- Portfolio equity curve, PnL distribution bar chart, drawdown chart
- Ground truth: accuracy vs horizon bar chart, RSI vs actual return scatter

3.7 LLM Interpreter (llm_interpreter.py)

OPTIONAL module. Uses HuggingFace Inference API (Qwen/Qwen2.5-72B-Instruct by default) to generate natural language explanations of signals. The LLM receives the structured signal data and explains it - it NEVER generates or modifies signals.

Falls back to template-based interpretation when HF_TOKEN is not set. The template produces Turkish/English explanations using the signal's reasons list.

4. User Interface - 9 Tabs

1. Analiz (Analysis)

Single-ticker analysis. Generates signal, shows candlestick chart with SMA overlays, RSI+MACD indicator panel, structured JSON output, and natural language interpretation. Displays signal, confidence, risk level, and current price.

2. Backtest

Strategy backtesting with configurable parameters. RSI buy/sell thresholds, SMA filter, volatility filter, holding period, evaluation period. Shows equity curve, signal map on price chart, and full metrics table with buy & hold comparison.

3. Karsilastirma (Compare)

Side-by-side comparison of 2-5 tickers. Shows signal, confidence, risk, price, RSI, SMA, MACD, volatility, and risk/reward for each ticker in a comparison table.

4. Explainability

Deep dive into why a signal was generated. Shows indicator breakdown table with bullish/bearish direction, vote count, decision rules, reasons list, and trade details (entry, SL, TP, shares, R/R).

5. AI Decision

Full pipeline: multi-ticker signal generation + backtest validation + portfolio execution. Shows decision table with backtest accuracy and Sharpe for each ticker, portfolio status, and per-decision explanations.

6. Evaluation Dashboard

Multi-ticker evaluation. Runs backtests on multiple tickers, shows combined equity curve, drawdown chart, signal distribution (BUY/SELL correct/wrong stacked bar), and per-ticker performance table.

7. Portfolio Sim

Stateful portfolio simulation. Execute trades based on signals, track open positions, view PnL distribution, equity curve, and trade history. Supports portfolio reset.

8. Ground Truth

The 'model ne dedi? gercek ne oldu?' tab. Evaluates every historical signal against actual price outcomes at 5/10/20 day horizons. Shows accuracy per horizon, per-ticker breakdown, calibration analysis, and detailed signal-vs-reality table.

9. Rehber (Guide)

Usage guide with signal rules, supported assets, and tab descriptions.

5. Technical Indicators - Detail

RSI(14)

Relative Strength Index with Wilder smoothing. Period: 14 days. Zones: >70 overbought, <30 oversold, >60 bullish, <40 bearish, else neutral.

SMA(20/50) Crossover

Simple Moving Average crossover. Golden cross: $SMA_{20} > SMA_{50}$ + price above both = STRONG_BULLISH. Death cross: $SMA_{20} < SMA_{50}$ + price below both = STRONG_BEARISH.

MACD(12,26,9)

Moving Average Convergence Divergence. $EMA_{12} - EMA_{26}$ = MACD line. 9-period EMA of MACD = signal line. Histogram = MACD - signal. Detects crossovers.

Bollinger Bands(20,2)

20-period SMA +/- 2 standard deviations. Price above upper = overbought. Price below lower = oversold.

Volume Ratio

Current volume / 20-day average volume. >1.5x = bullish confirmation, <0.5x = weak signal.

ATR(14)

Average True Range for volatility measurement. Used for stop-loss/take-profit distance calculation and position sizing.

Volatility Regime

Annualized volatility from 30-day returns. LOW: <15%, MODERATE: 15-30%, HIGH: 30-50%, EXTREME: >50%.

6. Signal Generation Rules

The signal is determined by majority vote among 5 indicators:

- IF 4+/5 indicators are bullish/oversold -> STRONG_BUY
- IF 3+/5 indicators are bullish/oversold -> BUY
- IF 4+/5 indicators are bearish/overbought -> STRONG_SELL
- IF 3+/5 indicators are bearish/overbought -> SELL

OTHERWISE -> HOLD

Volatility safety filter:

IF volatility regime = EXTREME AND signal = BUY/STRONG_BUY -> downgrade to HOLD

SELL signals are NOT filtered (selling in high volatility is acceptable).

7. Risk Management

Position sizing (ATR-based):

Conservative: 1% risk/trade, 1.5x ATR stop-loss, 2x ATR take-profit

Moderate: 2% risk/trade, 2x ATR stop-loss, 3x ATR take-profit

Aggressive: 4% risk/trade, 3x ATR stop-loss, 4.5x ATR take-profit

Portfolio limits:

Max single position: 20% of portfolio

Risk/reward ratio: always calculated (TP distance / SL distance)

Shares = $\text{risk_amount} / \text{SL_distance}$ (exact position sizing)

8. Data Sources & Fallback

Primary: yfinance (Yahoo Finance) for stocks, ETFs, indices, commodities, forex.

Secondary: Binance REST API for cryptocurrency (no API key needed, public endpoints).

Fallback: Built-in sample data generator using seeded random walk.

The fallback is critical: it guarantees the Space always works even when:

- Yahoo Finance is rate-limiting or down
- Binance is blocked in the region
- The user enters an unsupported ticker

Sample data uses numpy random with seed = hash(ticker) for consistency.

9. Supported Assets

Type	Format	Examples
US Stocks	TICKER	AAPL, NVDA, MSFT, GOOGL, TSLA, AMZN, META
BIST (Turkey)	TICKER.IS	THYAO.IS, GARAN.IS, AKBNK.IS
Crypto	TICKER-USD	BTC-USD, ETH-USD, SOL-USD
Commodities	Futures	GC=F (Gold), SI=F (Silver)
ETFs	TICKER	SPY, QQQ, IWM
Forex	XXX=X	USDTRY=X, EURTRY=X

10. Performance Metrics

Backtesting metrics: accuracy (%), total return (%), average return per trade, Sharpe ratio, max drawdown, profit factor, avg win, avg loss, buy accuracy, sell accuracy.

Portfolio metrics: equity, cash, total PnL (\$, %), open positions, total trades, win rate, avg win %, avg loss %, profit factor, max drawdown.

Ground truth metrics: per-horizon accuracy (5d, 10d, 20d), buy vs sell accuracy, average directional return, calibration (high vs low conviction accuracy).

11. Structured Output Format

Every signal is returned as structured JSON, not free text:

```
{
  "signal": "BUY",
  "confidence": 0.62,
  "reason": "RSI oversold at 28.5 -- potential bounce",
  "risk": "medium",
  "indicators": {
    "rsi": 28.5,
    "sma_crossover": "BULLISH",
    "macd": "BULLISH_CROSSOVER",
    "bollinger": "OVERSOLD",
    "volatility": "MODERATE"
  },
  "trade": {
    "entry_price": 230.50,
    "stop_loss": 222.40,
    "take_profit": 242.65,
    "shares": 25,
    "risk_reward": "1:1.5"
  },
  "explainability": { ... }
}
```

12. Limitations & Known Issues

- No real-time streaming: signals are point-in-time, not live-updated

13. Future Improvements / Roadmap

High Priority

- Persistent storage (SQLite/PostgreSQL) for portfolio state and signal history

Medium Priority

- Sentiment analysis integration: FinBERT on news headlines as 6th voting indicator

Exploratory / Long-term

- Reinforcement learning for position sizing optimization

14. Tech Stack

Runtime: Python 3.13 on Hugging Face Spaces

UI: Gradio 6.x (SSR disabled for Spaces compatibility)

Charts: Plotly 6.x (dark theme, interactive)

Data: yfinance 1.3+, Binance REST API (no key), numpy, pandas

Models: Pydantic 2.x (data validation)

LLM: HuggingFace Inference API (optional, Qwen/Qwen2.5-72B-Instruct)

Hosting: Hugging Face Spaces (free tier compatible)

Total codebase: ~118 KB across 13 files

No external databases. No API keys required for core functionality.

Zero-config deployment: git clone + pip install + python app.py